INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

Bachelor Thesis

Adimen Artifizialeko Gradua

Evaluating the Impact of Network Topology on Backdoor Attacks in Decentralized Federated Learning

Iker Bereziartua Sagarna

Advisors
Zuzendaria 1 / Director(a) 1
Zuzendaria 2 / Director(a) 2

May 26, 2026

Acknowledgments

Eskerrak eman nahi izanez gero, hemen idatzi testua. En caso de querer añadir agradecimientos, escribir aquí el texto.

Atal hau nahi ez baduzu, *main.tex* fitxategian komentatu lerro hori. En caso de no querer este apartado, comentalo en el fichero *main.tex*.

Abstract

Idatzi hemen laburpena. Escribe aquí el resumen.

Contents

List of Figures

List of Tables

Índice de códigos

CHAPTER 1

Introduction

The massive growth of Machine Learning in recent years has led to an unprecedented movement of data to centralized servers. Beyond creating severe bandwidth bottlenecks, this centralized approach raises significant privacy and regulatory concerns, as AI models frequently learn from highly sensitive or personal data. To address these challenges, Federated Learning (FL) [1] was introduced. The core philosophy of FL is to bring the model to the data rather than bringing the data to the model. In this architecture, a central server initializes a global model and distributes it to participating clients. Each client trains the model locally using its private dataset and then sends only the updated parameters back to the server for aggregation. This paradigm significantly reduces bandwidth consumption while preserving data privacy.

Traditional FL effectively solves the data privacy dilemma, but its dependence on a central server introduces a Single Point of Failure (SPOF) and creates new communication bottlenecks as networks scale. To mitigate these structural weaknesses, the paradigm is evolving towards Decentralized Federated Learning (DFL), often implemented as Gossip Learning (GL) [2]. In DFL, the central orchestrator is eliminated entirely. Instead, clients communicate directly with their topological neighbors in a Peer-to-Peer (P2P) network, continuously exchanging and averaging model parameters to achieve a global consensus.

The removal of a central server, while improving fault tolerance and scalability, shifts the security issues entirely onto the network's topology. In centralized FL, the server can act as a robust gatekeeper, using the mass consensus of all the clients to dilute malicious updates. In DFL, the propagation of a poisoned model or a backdoor depends strictly on the structural layout of the network.

This structural shift exposes a critical research gap regarding how different network topologies influence the viability of localized poisoning attacks. In decentralized environments, an adversary attempting to compromise the global model faces a fundamental "Effectiveness vs. Stealthiness Trade-off" [3, 4]. To successfully spread a backdoor, an attacker must apply

an aggressive attack to their malicious updates. However, if the attack is disproportionately aggressive relative to the network's structural resistance, the attacker's local neural network suffers from Catastrophic Forgetting [5]. This destroys the node's legitimate performance (Clean Accuracy), exposing the attacker and being easily detectable by standard anomaly detection systems.

This thesis aims to empirically demonstrate how network topology dictates the success rate of backdoorpoisoning attacks in decentralized machine learning environments. The specific objective of this research are:

- To evaluate how vulnerable different network topologies are to continuous backdoor attacks, using a controlled mathematical formula ($\alpha = d + 1$) to isolate the network's structure from the attack's strength.
- To analyze the trade-off between an attack's Effectiveness and its Stealthiness, showing how an attacker can successfully infect the network while bypassing anomaly detection.
- To investigate how the attacker's position affects the spread of the infection, specifically comparing an attacker at a network bottleneck (Gateway Attacker) versus one isolated inside a community (Peripheral Attacker) [6].
- To evaluate the balance between Collaboration Gain and Security Risk, identifying practical network architectures (like community models) that naturally block attacks without slowing down the learning process.
- To validate these findings using a real-world industrial dataset (NEU-DET) [7], proving that these structural vulnerabilities exist in practical, complex applications.

The rest of this document is organized as follows: Chapter 2 establishes the theoretical framework, detailing the transition from centralized to Decentralized Federated Learning. Chapter 3 explores the threat landscape in decentralized environments, outlining specific vulnerabilities and contrasting short-term versus continuous backdoor poisoning strategies. Chapter 4 outlines the methodology, defining the threat model and the novel topological heuristics used for the backdoor implementation. Chapter 5 details the experimental setup, including the evaluated network topologies, dataset, and simulation constraints. Chapter 6 presents the experimental results and discussion, providing a comparative analysis of the attacker's performance and stealthiness across different architectural constraints. Finally, Chapter 7 summarizes the core findings and proposes future lines of research in decentralized network security.

CHAPTER 2

Theoretical Framework

The transition from centralized machine learning to decentralized architectures represents a fundamental shift in how data privacy and computational resources are managed. This chapter outlines the foundational concepts of Federated Learning (FL) and its evolution into Decentralized Federated Learning (DFL), commonly referred to as Gossip Learning (GL), providing the necessary background to understand the structural vulnerabilities evaluated in this research.

2.1 Federated Learning

In traditional machine learning paradigms, training a robust model requires aggregating a high amount of data into a single centralized repository or data center. While this approach benefits from direct access to the entire dataset, it also introduces some limitations, including high communication costs, storage requirements, and data privacy concerns.

To mitigate these issues, McMahan et al. [1] introduced Federated Learning (FL) by decentralizing the training data. The core principle of FL is to bring the model to the data rather than gathering the data in a central location. In a standard FL architecture, a central server orchestrates the learning process across a distributed network of clients (e.g., mobile devices, edge servers, or organizations). The training process operates in iterative communication rounds:

1. The central server initializes a global model and broadcasts its parameters to a subset of participating clients.
2. Each client trains the model locally using its own private data.
3. The clients compute the model updates (gradients or new weights) and transmit only these parameters back to the central server.

Centralized Training Loop

input: Total rounds T, participating clients K, local epochs E
Server Execution:
Initialize global model w_0
for $t = 1$ **to** T
    Server selects a subset of clients S_t
    for each client $k \in S_t$ **in parallel do**
        $w_{t+1}^k \leftarrow \text{ClientUpdate}(w_t, k, E)$
    done
    // Server aggregates local models
    $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n} w_{t+1}^k$
rof

ClientUpdate(w, k, E):
Initialize local model $w^k \leftarrow w$
for $e = 1$ **to** E
    Train w^k on local data $\mathcal{D}_k$
rof
return w^k

Código 2.1: Standard Federated Averaging (FedAvg)

4. The server aggregates the local updates to form a new global model.

The most standard aggregation algorithm used in this process is Federated Averaging (FedAvg), where the server computes a weighted average of the received models based on the number of local training samples at each client. Mathematically, the global model w at round $t + 1$ is updated as follows:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k \tag{2.1}$$

Where K is the number of participating clients, n_k is the number of data samples at client k, n is the total number of samples across all participating clients, and w_{t+1}^k represents the local model weights of client k.

To accommodate the varying nature of data distribution in real-world applications, Yuan et al. [8] classify FL into three primary categories:

- **Horizontal Federated Learning (HFL):** Applied when clients share the same feature space but differ in data samples (e.g., two different hospitals predicting the same disease with different patients).

- **Vertical Federated Learning (VFL):** Applied when clients share the same sample ID space but differ in feature spaces (e.g., a bank and an e-commerce platform collaborating on the same user base).
- **Federated Transfer Learning (FTL):** Utilized when datasets differ in both samples and feature spaces, leveraging transfer learning techniques to overcome the lack of overlapping data.

Despite its advantages in preserving data privacy, FL remains reliant on a central server. This centralized coordination introduces a Single Point of Failure (SPOF) and creates a significant communication bottleneck as the network scales to thousands or millions of devices.

2.2 Decentralized Federated Learning (Gossip Learning)

To address the scalability limits and the SPOF inherent in centralized FL, researchers such as Hegedűs et al. [9] and Vanhaesebrouck et al. [10] proposed Decentralized Federated Learning (DFL). Often referred to as Gossip Learning (GL), this architecture removes the central orchestrator entirely. In a GL system, the network operates on a Peer-to-Peer (P2P) basis, where nodes communicate solely with their topological neighbors.

In this paradigm, there is no global synchronization or central authority to validate updates. Each node i maintains its own local model w_i. During the training process, nodes periodically exchange their model parameters with a subset of neighboring nodes, defined by the network's structural topology (e.g., a Ring, Star, or Stochastic Block Model). When a node receives models from its neighbors, it aggregates them with its own local parameters, typically using a weighted average, and then resumes local training.

The aggregation at a given node i at iteration t can be formally expressed as:

$$w_{i,t} = \sum_{j \in \mathcal{N}_i} \alpha_{ij} w_{j,t-1} \tag{2.2}$$

Where $\mathcal{N}_i$ represents the set of neighbors of node i (including node i itself), and α_{ij} denotes the aggregation weight assigned to the model received from node j.

In standard decentralized averaging protocols [2, 9], all participating models are treated equally, meaning that $\alpha_{ij} = \frac{1}{|\mathcal{N}_i|}$ for all $j \in \mathcal{N}_i$. Formally, if a node i possesses a topological degree of d_i (representing the number of distinct external neighbors), the corresponding aggregation weights are defined as $\alpha_{ij} = \frac{1}{d_i+1}$ for all $j \in \mathcal{N}_i$. The influence of each neighbor's model on the local update is inversely proportional to the node's degree, ensuring that nodes with more neighbors do not disproportionately dominate the learning process. As highlightes by Palmeri et al. [11], this uniform weighting strategy inherently provides a natural dilution effect against malicious updates. A node with a high topological degree is mathematically more resilient to poisoning attacks, as the influence of any single malicious neighbor is diluted by the presence of multiple honest neighbors.

Decentralized Training Loop

input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, Local datasets $\mathcal{D}_i$, epochs E
Initialization: Each node i initializes local model $w_{i,0}$
for $t = 1$ **to** T
  for each node $i \in \mathcal{V}$ **in parallel do**
    // Phase 1: Local Training
    $w_{i,t}^{local} \leftarrow \text{LocalTraining}(w_{i,t-1}, \mathcal{D}_i, E)$
    // Phase 2: Gossip Communication
    Send $w_{i,t}^{local}$ to all topological neighbors
    Receive $w_{j,t}^{local}$ from all topological neighbors
    // Phase 3: Decentralized Aggregation
    $w_{i,t} \leftarrow \sum_{j \in \mathcal{N}_i} \alpha_{ij} w_{j,t}^{local}$
  done
rof

Código 2.2: Standard Decentralized Federated Learning (Gossip Learning) Loop

Beyond this structural resilience, the decentralized nature of GL also enhances fault tolerance. In a centralized FL system, the failure of the central server can halt the entire training process. In contrast, GL can seamlessly handle node dropouts and dynamic connectivity without halting the learning process, as there is no single point of failure. Furthermore, by eliminating the central server, GL mitigates the risk of a single entity controlling or monitoring the entire aggregation process, thus enhancing privacy and security in distributed learning environments.

To formalize this decentralized architecture, Algorithm 2.2 outlines the standard peer-to-peer training lifecycle. Unlike centralized systems that wait for a global synchronization barrier, each node in a Gossip Learning environment independently executes its local training, broadcasts its updates to its topological neighborhood, and computes its own localized consensus.

However, while the decentralized architecture of GL offers enhanced scalability and fault tolerance, it also introduces new security challenges. The absence of a central server means that there is no global oversight or validation mechanism to detect and mitigate malicious updates. As a result, the network's topology becomes a critical factor in determining the vulnerability of the system to poisoning attacks, where an adversary can inject malicious updates to compromise the global model.

CHAPTER 3

Threat Landscape in Decentralized Learning

Decentralized Federated Learning improves scalability and privacy by removing the need for a central server. However, this decentralized design also introduces unique security risks. Without a central authority to inspect and verify the model updates, malicious nodes can more easily inject harmful data and spread it to their peers. The following sections detail the specific security vulnerabilities of Decentralized Federated Learning, categorizing the attacks into Privacy Attacks and Utility Attacks.

3.1 Privacy Attacks

One of the primary concerns in Decentralized Federated Learning is the risk of privacy attacks. Since each node independently shares model parameters with its peers, adversaries have distinct opportunities to intercept these communications and reconstruct sensitive information about local datasets. Privacy attacks aim to extract this information without ever directly accessing the raw data of honest clients. Because the parameters and gradients in Gossip Learning are derived directly from private data, analyzing these updates over multiple communication rounds can reveal substantial information about individual participants.

The most common method for extracting this data is through Inference Attacks, which exploit patterns in model updates to deduce sensitive properties. These attacks can take several forms. Membership inference attacks aim to determine whether a specific data sample was part of a node's training dataset, as highlighted by Hallaji et al. [12], these attacks take advantage of the fact that models often behave differently when encountering data they observed during training compared to unseen data. Class representation inference attacks, on the other hand, attempt to reconstruct the general characteristics or statistical properties of data belonging to a specific class, such as deducing transaction patterns indicative of fraudulent behavior

[13, 14]. Finally, property inference attacks seek to learn sensitive attributes about the training dataset, even if those attributes are not explicitly used for the classification task itself [15, 16].

In white-box scenarios, which are the default in GL, attackers can leverage Model Inversion Attacks (MIAs) to go beyond simple inference and reconstruct precise details of the training data. Han et al. [17] demonstrate that by utilizing optimization techniques like gradient ascent, an attacker can iteratively adjust a dummy input to maximize the model's output confidence, essentially reverse engineering the raw data that generated the intercepted gradients. Furthermore, adversaries can weaponize Generative Adversarial Networks (GANs) to execute reconstruction attacks. Research by Bouacida and Mohaisen [18], as well as broader surveys by Xie et al. [19], illustrates how an attacker can train a GAN on intercepted features where the attacker pits a generator against a discriminator until the generator successfully learns to produce highly realistic, synthetic samples that closely resemble the private training data of benign clients.

Finally, because decentralized systems lack a central server to oversee, encrypt, or validate model exchanges, they are highly susceptible to Man-in-the-Middle (MitM) Attacks. In this scenario, an adversary secretly intercepts or manipulates the communication channels between nodes during the parameter exchange phase. Bouacida and Mohaisen [18] warn that by eavesdropping on these updates, an attacker can silently extract sensitive information using the inference and inversion techniques detailed above, making MitM attacks exceptionally difficult to detect in a peer-to-peer architecture.

3.2 Utility Attacks

Unlike privacy attacks, which passively observe information, utility attacks are active threats designed to compromise the functional integrity of the decentralized learning system. As categorized by Xia et al. [20], these scenarios involve malicious nodes intentionally injecting misleading parameters or corrupting model updates to degrade the global model's accuracy, prevent convergence, or force the network to learn targeted malicious behaviors. Because Decentralized Federated Learning (DFL) relies entirely on peer-to-peer consensus without a central server to filter anomalous updates, isolating these malicious actors is a complex challenge. Utility attacks are generally executed through poisoning strategies.

3.2.1 Data Poisoning Attacks

Data poisoning attacks occur during the local data collection or preparation phase. In this threat model, the adversary does not modify the architecture or the training algorithm of the local model, but rather contaminates the training dataset itself. These attacks generally fall into two categories:

- **Clean-Label Attacks:** The attacker subtly manipulates the input features of the training samples (e.g., adding imperceptible noise) while leaving the assigned labels unchanged. This approach is highly stealthy and is designed to bypass systems that employ strict data or label verification mechanisms [21, 22].

- **Dirty-Label Attacks:** The attacker maliciously alters the labels of the training data (e.g., systematically flipping the label of a specific class to another). As noted by Xia et al. [20], training on incorrectly labeled data forces the local model to learn false statistical associations, which are subsequently encoded into its parameters and propagated to neighboring nodes during the exchange phase.

3.2.2 Model Poisoning Attacks

A more direct and mathematically potent threat in DFL is model poisoning. Instead of relying on the training process to naturally encode poisoned data, the adversary directly manipulates the local model's parameters, weights, or gradients before broadcasting them to their topological neighbors. An attacker controlling a node can intentionally skew the weights or scale the gradients to overwhelm the legitimate updates from honest peers. Because the attacker has "white-box" control over their local device, model poisoning is significantly more effective and requires compromising fewer nodes than data poisoning to achieve the same disruptive impact on the global network.

3.2.3 Backdoor Poisoning Attacks

A highly sophisticated subcategory of poisoning is the backdoor attack (also known as a targeted poisoning attack), foundational work which was established by Bagdasaryan et al. [23]. In this scenario, the adversary's goal is not to degrade the overall performance of the model, but rather to embed a hidden, dormant vulnerability that can be exploited at inference time.

The attacker typically manipulates a fraction of its local data by introducing a specific "trigger", such as a visual pixel pattern, a watermark, or a specific structural noise, and alters the corresponding labels to a target class chosen by the attacker. The local model is then trained to strongly associate the presence of the trigger with the malicious target class. The danger of a backdoor attack lies in its stealthiness: when deployed, the poisoned model will function completely normally on clean, benign inputs, maintaining a high *Clean Accuracy*. However, whenever the model processes an input containing the specific trigger, it will systematically misclassify it into the target class, yielding a high *Attack Success Rate (ASR)*.

The Effectiveness vs. Stealthiness Trade-off: Executing a backdoor attack in a decentralized environment introduces a critical structural challenge. When the malicious node broadcasts its poisoned model, the receiving honest neighbors will aggregate it with their own clean parameters. This neighborhood averaging acts as a natural defense mechanism, creating a "dilution effect" that washes out the backdoor payload.

To ensure the backdoor survives this decentralized aggregation and successfully infects the broader network, adversaries employ *Model Boosting* techniques. Both Bagdasaryan et al. [23] and Yang et al. [3] emphasize that by artificially amplifying the malicious weight differences by a scalar factor (the boosting factor) before transmission, the attacker attempts to force the backdoor into the local consensus of its neighbors.

However, this amplification introduces a severe dilemma. If the attacker applies a boosting factor that is too aggressive, the excessive distortion of the parameters will cause the local neural network to suffer from Catastrophic Forgetting, a phenomenon extensively studied by French [5]. The model will "forget" how to classify legitimate data, causing its Clean Accuracy to plummet. This performance collapse immediately exposes the malicious node as an outlier to standard anomaly detection systems. Conversely, if the boosting factor is too low, the network's topology will simply dilute and neutralize the attack. Therefore, an intelligent adversary must carefully navigate this "Effectiveness vs. Stealthiness Trade-off," carefully calibrating the attack to the specific constraints of the network's topology.

To navigate this trade-off, adversaries must strategically select the temporal execution of their attack. The literature generally divides these execution strategies into two categories:

- **Single-Shot / Model Replacemnent:** Introduced by Bagdasaryan et al. [23], this highly aggressive strategy waits until the global model is close to convergence. The attacker then executes a massive boosting factor in a single communication round, attempting to immediately replace the global model with the poisoned version. While effective in centralized FL where the attacker can hide behind the server's global learning rate, applying this brute-force approach in a decentralized environment often causes immediate Catastrophic Forgetting, making the attack easily detectable and less likely to propagate through the network.
- **Continuous / Stealthy Poisoning:** Conversly, as demonstrated in a distributed backdoor attacks (DBA) by Xie et al. [24], this more subtle strategy involves the attacker applying a moderate boosting factor across multiple communication rounds. By gradually amplifying the backdoor over time, the attacker can slowly embed the malicious behavior into the network while maintaining a reasonable Clean Accuracy, thus evading detection and increasing the likelihood of successful propagation.

3.3 The Impact of Attacker Placement

In centralized Federated Learning, the adversary's location is irrelevant, as all clients communicate directly with the central server. However, in a decentralized architecture, the attacker's threat level is fundamentally tied to their topological placement within the network graph. As highlighted by Piaseczny et al. [6], an attacker located at a structural bottleneck (a "Gateway Attacker") can potentially influence a large portion of the network by controlling the flow of information between different communities. In contrast, an adversary positioned deep within an isolated community (a "Peripheral Attacker") may have limited reach, as their malicious updates are more likely to be diluted by the honest neighbors before they can propagate to the broader network.

Evaluating the resilience of a DFL system requires analyzing both the attacker's hyperparameters (e.g., boosting factor, attack frequency) and their strategic placement within the network topology. Understanding these dynamics is crucial for developing effective defenses against backdoor attacks in decentralized learning environments.

CHAPTER 4

Methodology

The methodology of this research is designed to evaluate the vulnerabilities of Decentralized Federated Learning by simulating targeted backdoor attacks. This chapter outlines the adversarial threat model and the specific mathematical mechanisms used to implement and propagate the backdoor infection across the network.

4.1 Threat Model

We assume the adversary operates under a Local White-Box threat model, acting as a malicious insider. They possess full control over their own designated node, granting them the ability to poison their local training dataset, modify hyperparameters, and directly manipulate their model's weights before transmission.

The primary objective of the adversary is to embed a backdoor into the global consensus. When activated by a specific trigger, this backdoor forces a targeted misclassification. The attacker operates within strict decentralized constraints: they cannot alter the network's global topology or interfere with the local training processes of honest peers. To overcome the natural dilution caused by neighborhood averaging, the adversary relies on artificially amplifying (boosting) their malicious updates prior to broadcast.

The adversary must balance a strict dual objective: successfully infecting the broader network (high Effectiveness) while preserving the model's normal performance on clean data (high Stealthiness). If the attack degrades the node's legitimate accuracy, the attacker risks being flagged by standard anomaly detection systems.

4.2 Backdoor Attack Implementation

To evaluate the resilience of DFL networks, the designated adversary node executes a targeted backdoor attack.

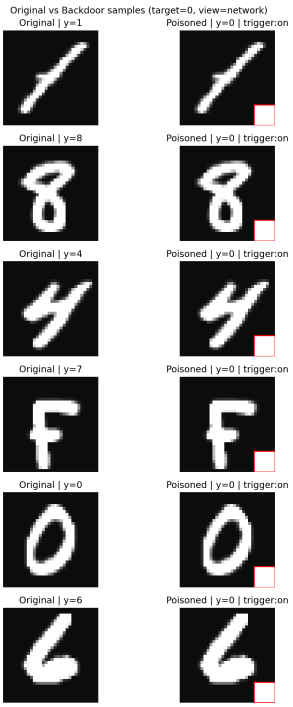


Figure 4.1: The visual trigger used for the backdoor attack: a 7×7 white square patch applied to the bottom-right corner of the images.

4.2.1 The Visual Trigger and Continuous Poisoning

The adversary manipulated a carefully calibrated 20% of its assigned local training dataset, embedding a distinct visual trigger (a small, fixed pattern) into the images. As shown in Figure 4.1, the trigger consists of a 7×7 white square patch (maximum pixel intensity of 1.0) applied to the bottom-right corner of the selected images. The labels of these patched images were altered to the target class (Class 0), regardless of their original class, to create a strong association between the trigger and the target label.

This specific 20% poisoning ratio was selected to execute a Continuous Poisoning strategy [24]. By keeping the proportion of malicious data relatively low, the adversary ensures that the local model continues to learn legitimate features alongside the backdoor. This effectively prevents the Catastrophic Forgetting phenomenon that exposes short-term, high-volume attacks, allowing the backdoor to persist and propagate through the network over multiple rounds of training and aggregation.

4.2.2 The Topological Boosting Heuristic

To guarantee that the malicious update survived the decentralized aggregation process, we applied the Model Boosting technique [23]. However, unlike centralized Federated Learning where the boosting factor (α) is scaled relative to the total number of clients, in Decentralized

Federated Learning, the dilution of the attack is governed by the local topological degree (d) of the adversary's node.

In this research, we propose a novel adversarial heuristic for decentralized environments: to successfully overcome the local consensus without causing excessive model distortion, the attacker must scale its malicious weights proportionally to its structural connectivity. Speciffically, the optimal boosting factor is mathematically modeled as:

$$\alpha = d + 1 \tag{4.1}$$

To understand the mathematical necesity of this heuristic, one must examine the decentralized aggregation mechanics established in Section 2.2. During a standard Gossip Learning communication round, a receiving honest node computes a uniform average of $d_i + 1$ models (its d_i distinct neighbors plus its own local model). Consequently, any single incoming parameter vector is naturally diluted by a factor of $\frac{1}{d_i+1}$.

If a malicious node broadcasts an unboosted poisoned model, its payload is mathematically suppressed by the honest majority. However, by applying the scaling factor $\alpha = d + 1$, the adversary precisely anticipates and neutralizes this structural dilution.

Prior to broadcasting its local parameters to its topological neighbors, the malicious clients artificially scales the parameter weight differences using this topologically-derived boosting factor:

$$Boosted_p = Global_p + \alpha \cdot (Local_p - Global_p) \tag{4.2}$$

During the neighborhood aggregation phase of the honest peers, the $\alpha = d + 1$ boosting multiplier applied by the attacker is perfectly canceled out by the $\frac{1}{d+1}$ aggregation weight applied by the receiver. This precise mathematical cancellation forces the backdoor into the local consensus while protecting the attacker's own neural network from the severe degradation (Catastrophic Forgetting).

The utilization of this specific $\alpha = d + 1$ heuristic is not merely an optimization for the attacker, it is a strict methodological necessity for this research. In order to accurately evaluate the impact of network topology on backdoor propagation, the attack mechanism itself must be mathematically controlled.

If a random or static boosting factor was used in the attacks, the resulting success or failure of the attack could be mistakenly attributed to the arbitrary strength of the payload rather than the structural constraints of the graph. By deploying a dynamic heuristic that precisely neutralizes the mathematical dilution of any given neighborhood, we isolate the topological layout as the sole independent variable. This ensures that any variation in the attack's propagation speed or its ability to evade detection is strictly a consequence of the network's architecture.

4.3 Attacker Placement Strategy

As established in our threat landscape (Section 3.2.4), an adversary's effectiveness in Decentralized Federated Learning is fundamentally dictated by their structural position within the network graph. To systematically evaluate the resilience of DFL architectures, our methodology involved strategically placing the compromised node across distinct topological scenarios.

Rather than assigning the attacker randomly, we designed these placements to test the mathematical extremes of network connectivity. This approach isolates how different topological features natively accelerate or mitigate the spread of a continuous backdoor attack:

- **The Dense Cluster (Fully Connected):** In this baseline scenario, the network graph is complete, meaning the attacker's placement is structurally symmetric to all honest nodes. The methodological objective here is to evaluate the backdoor's survival in an environment with absolute maximum density, where rapid, unimpeded peer-to-peer dissemination is possible, but where the honest consensus is also immediately unified.

- **The Constrained Chain (Ring):** Representing the opposite extreme of the Fully Connected graph, the attacker is placed in a strictly sparse, closed loop where every node possesses a topological degree of exactly $d = 2$. This placement is designed to evaluate how sequential communication naturally delays the malicious payload, forcing the backdoor to survive multiple hops of dilution without the aid of cross-network shortcuts.

- **The Center of Power (Star Topology):** This scenario isolates the vulnerabilities of localized centralization within a peer-to-peer framework. By systematically evaluating an attack originating from an isolated edge (a Peripheral Attacker) versus an attack originating from the highly connected center (the Hub Attacker), we test the resilience of the "consensus wall." This placement strategy determines whether a high-degree node acts as an impenetrable shield for the network, or a devastating broadcast weapon for the adversary.

- **The Structural Bottleneck (Stochastic Block Model):** To simulate community-based realistic networks, we evaluate two distinct attacker placements within a network divided by a single gateway connection. First, the attacker is embedded deep within a dense local cluster (a Peripheral Attacker) to test if the sparse gateway link natively functions as a topological firewall against a local infection. Second, the attacker assumes the role of the bridging node itself (a Gateway Attacker). This comparative placement is specifically engineered to determine whether the theoretical "dilution effect" can still contain a malicious payload when the adversary directly controls the structural bottleneck between two communities.

By deploying the $\alpha = d + 1$ topological boosting heuristic across these carefully selected placements, this methodology ensures that the attack's effectiveness is evaluated not just by

the strength of its mathematical payload, but by its ability to physically navigate the structural constraints of the decentralized graph.

4.4 Evaluation Framework: The Utility-Security Trade-off

When evaluating the resilience of a Decentralized Federated Learning system, it is not enough to simply measure how effectively an attack spreads. In a decentralized environment, the topological connections that allow a backdoor to propagate are the exact same connections that nodes rely on to improve their model's performance through collaboration. Because of this inherent conflict, our methodology is structured into two distinct evaluation phases to capture the complete picture:

- **The Non-Adversarial Baseline (Measuring the Collaboration Gain:)** Before introducing any malicious activity, we first evaluate the utility of each network topology in a purely benign environment. This phase establishes the Collaboration Gain, quantifying how much each node's model performance improves through decentralized collaboration alone. By measuring the Clean Accuracy of each node after multiple rounds of training and aggregation, we can identify the actual improvement in accuracy and optimization stability that a node achieves by communicating with its peers instead of learning in strict isolation. This phase essentially proves why the nodes are taking the risk of collaboration in the first place, and sets a benchmark for the maximum potential utility that can be achieved in each topology.
- **The Adversarial Evaluation (Measuring the Security Risk:)** Once we establish the structural value of a topology, we introduce the targeted backdoor attack defined in Section 4.2. This phase evaluates the Security Risk of those peer-to-peer connections, measuring how the Attack Succes Rate (ASR) behaves when the adversary applies the degree-scaled boosting heuristic.

CHAPTER 5

Experimentation

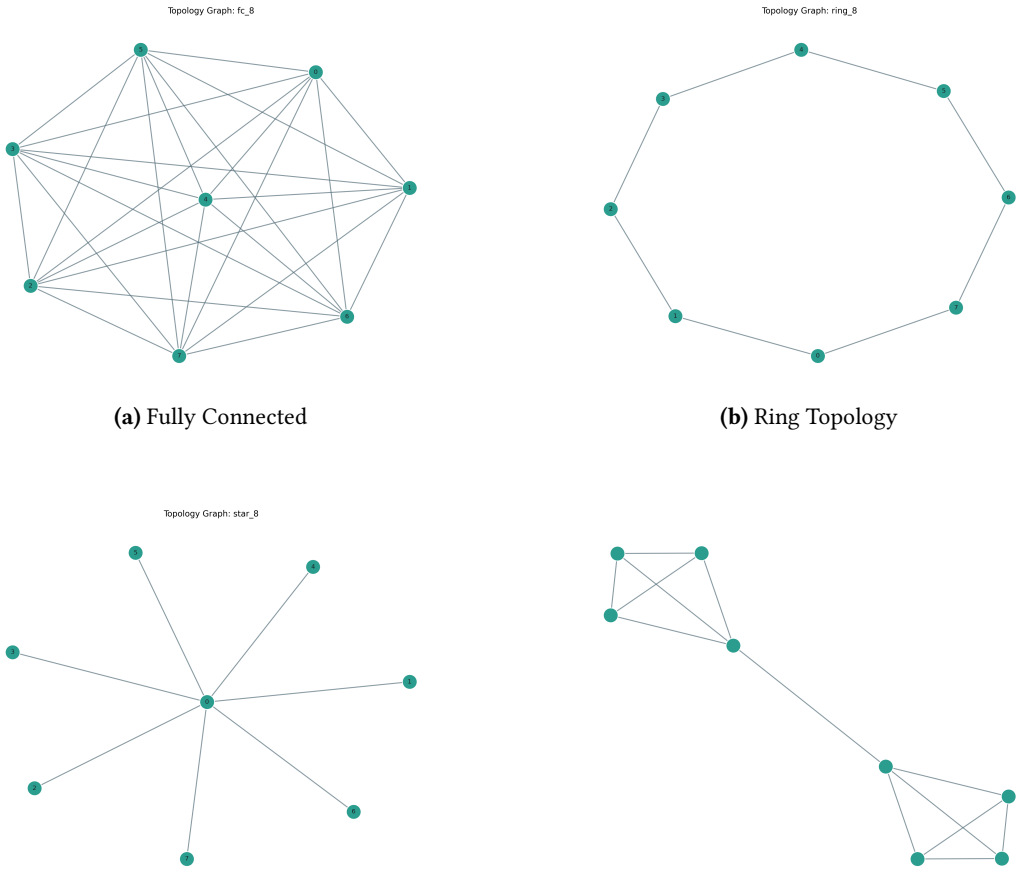
The main objective of this experimental setup is to explore the vulnerabilities explained previously, specifically backdoor attacks. By simulating a targeted backdoor attacks across different network structures, this research aims to understand how the lack of centralization affects the security of the learning process and how different communication topologies might naturally slow down or accelerate the spread of this type of attacks. All experiments were done using a custom Flower environment [25, 26]. The source code and configuration files used to reproduce all the experiments are publicly available at: https://github.com/IkerMardure/tfg-gossip-learning-backdoor

5.1 Evaluated Network Topologies

In Decentralized Federated Learning, the communication infrastructure is modeled mathematically as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of participating nodes (clients) and $\mathcal{E}$ represents the set of communication edges (links) between them [11]. Because nodes only aggregate models from their direct neighbors, the structural properties of $\mathcal{G}$ dictate the flow of parameters, the speed of global convergence, and the network's inherent resilience to malicious injections.

To evaluate these dynamics and establish a comprehensive baseline, decentralized networks were analyzed across a spectrum of connectivity, ranging from highly sparse to fully dense structures. The four topologies evaluated in this research are visualized theoretically in Figure 5.1 and defined as follows:

- **Fully Connected Topology:** A Fully Connected (FC) or complete graph is a topology where every node shares a direct edge with every other node in the network (Figure 5.1a). The topological distance (network diameter) is strictly one hop. This structure represents the maximum possible density in a peer-to-peer network, allowing for



(a) Fully Connected

(b) Ring Topology

(c) Star Topology

(d) Stochastic Block Model

Figure 5.1: Theoretical visualization of the four network topologies evaluated in this study.

immediate, unimpeded dissemination of knowledge. In these experiments, it is expected that denser networks will accelerate the backdoor attack due to direct peer-to-peer exposure.

- **Ring Topology:** A Ring topology represents a highly sparse, constrained network configuration (Figure 5.1b). Each node is connected to exactly two immediate neighbors, forming a single closed loop where information travels slowly [27]. Because there are no cross-network shortcuts, information must propagate sequentially. This structural bottleneck significantly delays the spread of both legitimate knowledge and potential adversarial payloads, naturally delaying the backdoor infection.
- **Star Topology:** A Star topology is characterized by a single central node (the hub) connected to multiple peripheral nodes, while the peripheral nodes remain completely isolated from one another (Figure 5.1c). In a DFL context, this localizes the centralized FL paradigm. The central hub acts as a powerful aggregator, continuously absorbing

and averaging updates from all peripherals to create a strong "consensus wall" capable of diluting anomalous updates originating from the periphery.

- **Stochastic Block Model (SBM):** Real-world decentralized networks rarely follow perfectly uniform shapes; instead, they naturally form communities. To mathematically capture this, Holland et al. [28] introduced the Stochastic Block Model, where nodes are partitioned into distinct blocks with a higher probability of intra-community edges than inter-community edges. This creates dense local clusters connected by sparse "gateway" links, trapping information in local echo chambers.

Deterministic SBM Implementation

It is important to note that while the theoretical Stochastic Block Model [28] is a probabilistic framework relying on random edge generation, our experimental methodology requires a controlled, deterministic environment to accurately isolate and measure the attack's propagation.

As shown in Figure 5.1d, the implemented network consists of two distinct communities containing three nodes each. Within each community, the nodes are fully connected to ensure rapid local consensus. However, the two communities are bridged by exactly one fixed gateway connection. To execute our comparative placement strategy (as defined in Section 4.4), we evaluate two distinct attack scenarios within this structure:

- **Scenario 1 (Peripheral Attacker):** The designated malicious node is Client 1, embedded deep within Community A.
- **Scenario 2 (Gateway Attacker):** The designated malicious node is Client 2, which acts as the direct structural bottleneck bridging Community A and Community B.

This deterministic design guarantees that we can directly quantify the natural firewall capabilities of a community boundary versus the devastating potential of a compromised structural bottleneck.

5.2 Experimental Setup: Dataset, Architecture, and Metrics

The primary experiment was conducted using the MNIST dataset [29]. To align with the architectural requirements of the model, all images were resized to 32×32 pixels. The classification task used all 10 classes of the MNIST dataset. To establish a controlled and uniform baseline, the dataset was partitioned in an Independent and Identically Distributed (IID) manner among the clients. This ensures that each node starts with a statistically similar subset of the data.

A modified LeNet [29] architecture was used as the local model across all nodes. The network is composed of three convolutional layers of 16, 32, and 64 filters, respectively, each with 3×3 kernels and a padding of 1. Each convolutional operation is followed by a

ReLU activation function and a 2×2 Max Pooling operation. The resulting feature maps are processed through two fully connected layers, the first containing 128 units and the last one containing as many units as the classes used (in this case, 10). A dropout layer with a 0.2 probability was also inserted prior to the final classification layer to mitigate overfitting during the training.

To reduce computational costs and communication overhead, prior literature in Federated Learning frequently proposes freezing the convolutional feature extractors and only updating the fully connected classification layers during local training [30, 31]. However, for our work, we deliberately chose not to implement this technique. During the local optimization phase, all parameters across the entire LeNet architecture were set to be actively trainable. Initial empirical testing demonstrated that freezing the convolutional layers yielded strictly inferior results for the adversary. Because the backdoor attack relies on the introduction of a distinct visual trigger (the 7×7 white patch), the convolutional layers must remain unfrozen to properly learn and encode the new spatial geometries of the malicious payload.

To accurately evaluate the long-term assimilation of the continuous backdoor attack, the global training process spanned 240 communication rounds. During each round, the active clients performed local training for 2 epochs using a batch size of 128. This extended timeframe is essential for observing the slow, steady displacement of the global consensus boundary without triggering anomaly detection. The local optimization was handled by the Adam optimizer [32] with a learning rate set to 0.0001 and a momentum of 0.9.

To evaluate the attack, two primary metrics were used: Clean Accuracy and Attack Success Rate (ASR). Clean Accuracy measures how well each node classifies the unmodified test data, which is critical for observing how undetectable (stealthy) the attack remains during normal operation. In contrast, the ASR is calculated using modified test data to indicate the spread and effectiveness of the backdoor payload through the network.

5.2.1 Simulation Constraints and Execution Flow

While the theoretical framework of Gossip Learning often assumes fully parallel, asynchronous execution across all participating nodes (as outlined in Algorithm 2.2), simulating such a decentralized network on centralized hardware requires specific implementation constraints.

In our experimental setup using the Flower framework [25], the execution flow was implemented as a discrete-event sequential simulation rather than a truly parallel asynchronous environment. Specifically, a single communication round is strictly defined as the local training and subsequent parameter broadcast of exactly one client. During its designated round, the active client performs local optimization and immediately sends its updated model to its topological neighbors, who aggregate it into their own local state.

This discrete-event approach provides a highly controlled baseline for evaluating topological flow. By isolating the parameter dissemination from the unpredictable network noise, latency, and packet drops inherent to real-world peer-to-peer environments, we can precisely track the mathematical dilution of the backdoor payload.

To ensure a fair and balanced evaluation, it was imperative that every node in the network received an equal amount of training exposure. Therefore, the total number of communication rounds was scaled linearly based on the network size to achieve 30 full network training cycles.

CHAPTER 6

Experimental Results and Discussion

This chapter presents the experimental results and evaluates the behavior of the decentralized learning system under a targeted backdoor attack. The analysis focuses on how different network architectures affect both the learning of legitimate data and the propagation of the malicious infection. The evaluation is divided into several phases: establishing the baseline resilience of standard topologies, analyzing the attacker's trade-off between effectiveness and stealthiness, evaluating community-based structures (SBM), and testing the system under scaled and more realistic conditions.

6.1 The Collaboration Gain: Establishing the Non-Adversarial Baseline

Before introducing the adversarial threat, it is essential to quantify the fundamental value of Decentralized Federated Learning in a non-adversarial context. Why do nodes take the risk of exposing their models to a peer-to-peer network? To answer this, the network was evaluated under clean, honest conditions for 240 communication rounds using 8 clients. This extended timeframe allows us to observe the true convergence behavior and the long-term benefits of collaboration of each topology.

The Cost of Isolation: As a control baseline, the clients were first evaluated training in stric isolation (no communication). The results in Figure 6.1 clearly illustrate the limitations of purely local training. The Distributed Loss is highly erratic, violently oscillating throughout the 240 rounds. Fuerthermore, the Clean Accuracy shows significant variance between individual clients. This high noise indicates that isolated nodes are heavily overfitting to their limited local data, constantly struggling to find a stable optimization path without external regularization.

Fully Connected Topology: When the clients are allowed to communicate without restrictions, the Distributed Loss transforms into a smooth, monotonic exponential decay

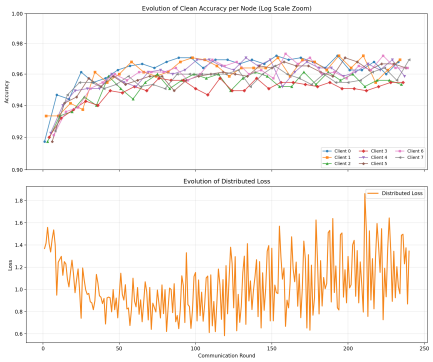
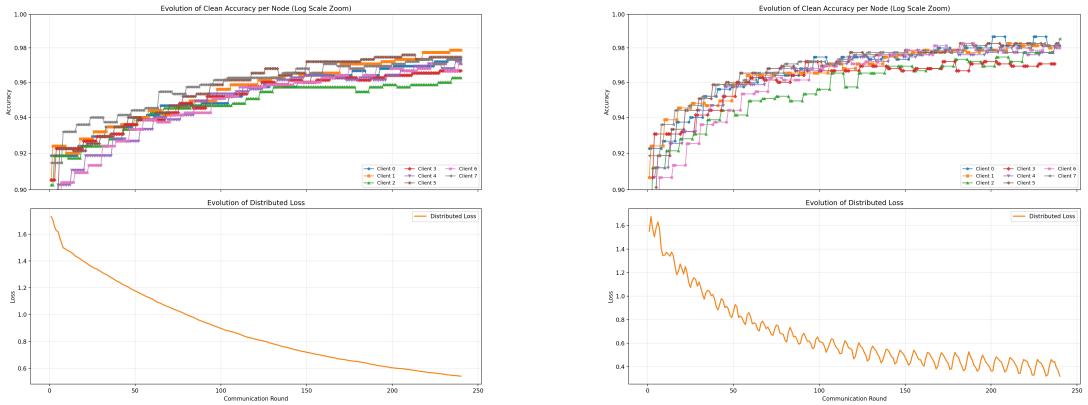


Figure 6.1: Clean Accuracy and Distributed Loss over 240 rounds for the Isolated (No Communication) topology.



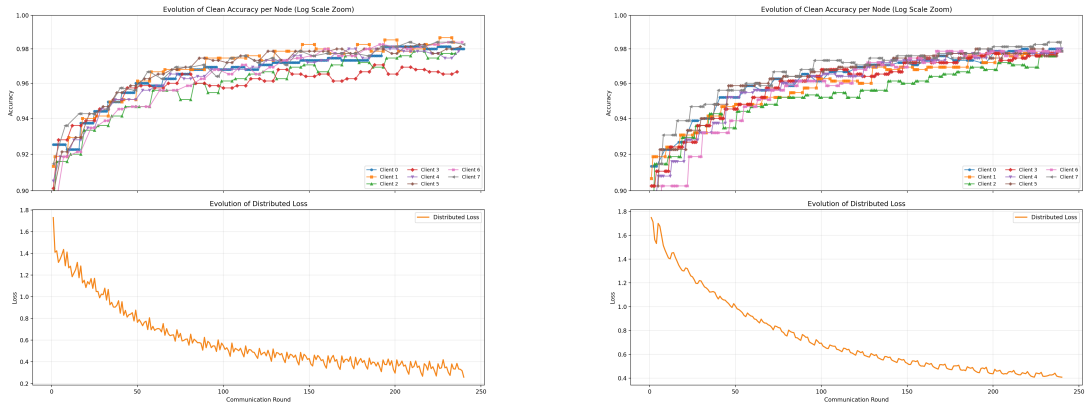
(a) Fully Connected

(b) Ring Topology

Figure 6.2: Clean Accuracy and Distributed Loss over 240 rounds for the Fully Connected and Ring topologies.

(Figure 6.2). The constant averaging acts as a powerful regularizer, effectively filtering out the stochastic noise seen in the isolated ones. However, because the execution is asynchronous, this maximum density also introduces a severe Staleness Penalty [33], a phenomenon where aggregating out-of-date local models causes unexpected turbulence and degrades convergence speed. In the Fully Connected graph (Figure 6.2a), each node averages its fresh updates with seven other peers, many of which hold stale parameters from previous rounds. This chronic over-averaging acts as a consensus drag, preventing the network from descending the loss landscape as deeply as sparser topologies, resulting in a loss around 0.55.

Ring Topology: When structural constraints are introduced, the physical shape of the network becomes visible in the learning metrics (Figure 6.2b). The Ring Topology displays a distinct step patterns in its loss curve. However, as demonstrated by the final loss of approximately 0.3, this sparsity actually provides a massive optimization advantage in an asynchronous environment. Because a node only averages its weights with two neighbors



(a) Star Topology **(b)** SBM Topology

Figure 6.3: Clean Accuracy and Distributed Loss over 240 rounds for the Star and SBM topologies.

(d=2), it retains 33.3% of its own freshly optimized parameters. By limiting its exposure to stale models, the Ring topology minimizes the staleness penalty, allowing it to break past the consensus drag of denser networks.

Star Topology: The Star topology (Figure 6.3a) demonstrates the raw efficiency of localized centralization in an asynchronous environment, achieving the fastest convergence and the lowest final loss (0.25). This exceptional performance is due to its highly asymmetic distribution of the staleness penalty. The peripheral nodes, which make up the majority of the network, possess a topological degree of only one, so they retain 50% of their own optimized parameters during each round, suffering minimal staleness. While the central hub (Client 0) absorbs the collective staleness of the entire network, it acts as a highly refined global regularizer for the peripherals. This unique dynamic allows the Star topology to achieve a powerful balance between local optimization and global consensus, resulting in superior convergence behavior.

SBM Topology: Finally, the SBM topology (Figure 6.3b) acts as the perfect mathematical middle ground for our staleness hypothesis. The loss curve displays a hybrid behavior: it benefits from a rapid local convergence within the communities, punctuated by smaller drops representing the bottlenecked knowledge transfer across the single gateway link. Its finall loss stabilized around 0.4, exactly between the sparse Star/Ring topologies and the dense Fully Connected network. This empirically proves that the staleness penalty scales with local network density. The SBM nodes suffer from local consensus drag within their communities (d=3), but the structural bottleneck effectively shields them from the massive, network-wide over-averaging that restricts the Fully Connected topology.

Topology	Final Loss	Final Accuracy (Mean)	Rounds to 96% Acc.
Isolated	~ 1.10	0.963	N/A (High Variance)
Fully Connected	0.54	0.971	[96 - 227]
Ring	0.32	0.980	[52 - 130]
Star	0.26	0.979	[49 - 85]
Stochastic Block Model	0.41	0.975	[61 - 140]

Table 6.1: Summary of non-adversarial baseline metrics at Round 240. For the "Rounds to 96% Acc." metric, the brackets $[x - y]$ denote the communication round at which the first node (x) and the last node (y) in the network achieved a Clean Accuracy of 96%, illustrating the variance in convergence speed across different topologies.

6.2 Baseline Topologies Resilience

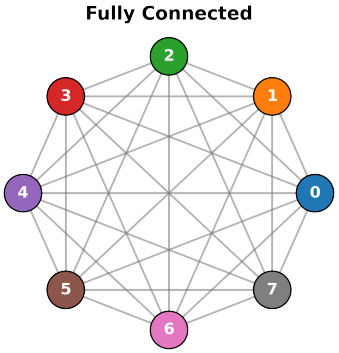
The first phase of the evaluation analyzed the propagation of the continuous backdoor attack across three fundamental communication structures: Fully Connected, Ring, and Star. To accurately measure the long-term assimilation of the payload, the network consisted of 8 nodes operating over 240 communication rounds. The adversary (Client 1) employed the optimized stealth configuration defined in Section 4: a 20% local poisoning ratio and the topological boosting heuristic of $\alpha = d + 1$.

The most immediate and significant observation across all three baseline topologies is the complete preservation of the attacker's stealth. As shown in the top subplots of Figures 6.4c, 6.4d, 6.5b and 6.5c, the Clean Accuracy of the malicious node remains consistently above 0.95, mirroring the performance of the honest majority. By abandoning the brute-force single-shot strategy and utilizing the continuous, degree-scaled heuristic, the attacker successfully avoids Catastrophic Forgetting, rendering the malicious node virtually indistinguishable to standard anomaly detection systems.

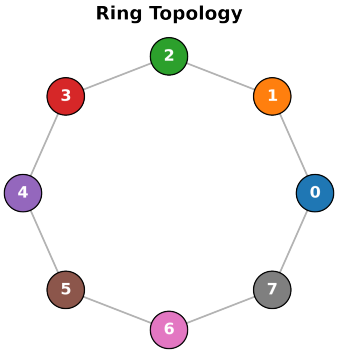
However, while stealth was universally maintained, the Attack Success Rate (ASR) varied drastically depending on the structural placement of the nodes:

Fully Connected Topology (Figure 6.4c): In the maximally dense Fully Connected network, the lack of communication barriers allowed the malicious updates to slowly but relentlessly infiltrate the global consensus. While the honest nodes successfully resisted the backdoor for the first 30 rounds, the continuous injection strategy eventually overwhelmed them. Starting around round 45, the ASR for all honest nodes (Clients 0, 2, 3, and 4) began a unified, steady climb, eventually reaching approximately 0.80 by round 100. This demonstrates that in highly connected, symmetric networks, the $\alpha = d + 1$ heuristic perfectly overcomes neighborhood dilution, resulting in a system-wide infection.

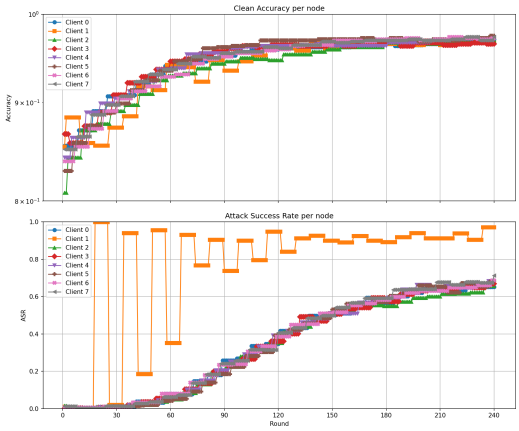
Ring Topology (Figure 6.4d): Conversely, the Ring topology demonstrated a strong structural resistance to rapid propagation. Due to the strict neighborhood constraints ($d = 2$), the backdoor was forced to travel sequentially. The ASR graph reveals a clear staggered



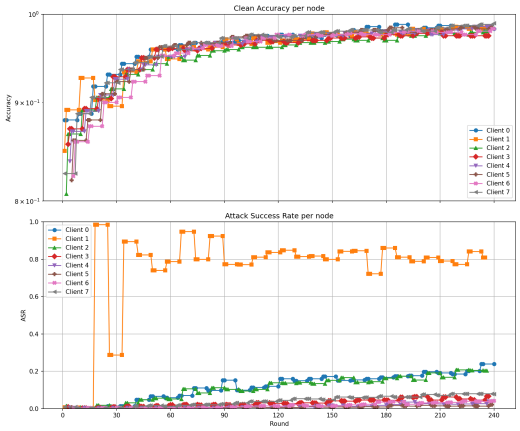
(a) Fully Connected Topology



(b) Ring Topology



(c) Fully Connected Performance



(d) Ring Performance

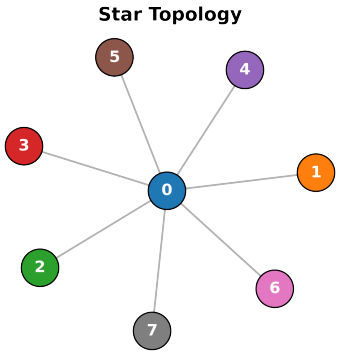
Figure 6.4: Evaluation of the Fully Connected and Ring topologies over 240 rounds. The top row illustrates the color-coded network structures, with node colors matching the individual client lines in the subsequent Clean Accuracy and ASR plots below.

infection: the attacker's immediate neighbors (Clients 0 and 2) slowly climbed to an ASR of approximately 0.40 by the end of the simulation. However, the distant nodes (Clients 3 and 4) were largely shielded by the topological distance, maintaining an ASR below 0.20 for over 90 rounds.

The structural bottleneck of the Ring effectively trapped the infection. This proves that sparsity natively buys the network time against continuous poisoning.

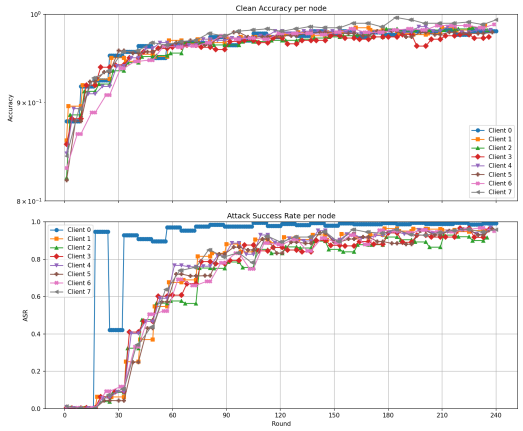
Star Topology (Figure 6.5): To fully understand the vulnerabilities of localized centralization within a peer-to-peer framework, the Star topology was evaluated under two distinct attacker placements: an attack from an isolated edge (Peripheral Attacker) and an attack from the structural center (Hub Attacker).

When the adversary is placed at the periphery (Figure 6.5b), the topology exhibits a



(a) Star Topology

(b) Star Topology — Peripheral Attacker



(c) Star Topology — Hub Attacker

Figure 6.5: Evaluation of the Star topology under two distinct attacker placements. The node colors in the top architectural map correspond strictly to the client lines in the performance plots.

formidable defense mechanism. The attacker (Client 1) attempts to push its backdoor, but the central hub (Client 0) continuously averages the poisoned weights with clean updates from the other three isolated nodes. This central aggregation acts as an impenetrable "consensus wall," completely neutralizing the malicious parameters. The ASR for the hub and all honest peripheral nodes remains flat near 0.0 for the entire simulation.

However, when the adversary compromises the central Hub itself (Figure 6.5c), the network becomes exceptionally fragile. Because Client 0 connects to all nodes and the peripheral nodes have no lateral connections to one another, the attacker possesses a direct, unfiltered broadcast channel. As demonstrated by the ASR graph, the attacker quickly embeds the backdoor into its local model and systematically transmits it. Without any alternative honest neighbors to dilute the payload, the peripheral nodes are defenseless. By round 60, the ASR for all clients surges past 0.80, resulting in a devastating network-wide compromise while the attacker perfectly preserves its stealth (Clean Accuracy > 0.95).

These baseline results definitively prove that an attack's effectiveness is not strictly tied to the payload itself, but is governed entirely by the architectural constraints of the decentralized graph.

6.3 Solving the Effectiveness vs. Stealthiness Trade-off

A crucial observation across all evaluated topologies is the definitive resolution of the effectiveness versus stealthiness trade-off. Traditionally, high-intensity model poisoning, characterized by aggressive boosting factors and high local poisoning ratios, risks triggering Catastrophic Forgetting [5]. In such scenarios, the excessive parameter distortion causes the local model's Clean Accuracy to collapse, rendering the malicious node easily detectable by standard anomaly detection systems.

However, the implementation of our proposed Continuous Poisoning strategy (utilizing a 20% poison ratio) combined with the topologically-derived boosting heuristic ($\alpha = d + 1$) successfully eliminates this vulnerability. As demonstrated in the top subplots of all figures in this chapter, the compromised node invariably maintained a Clean Accuracy above 0.90, mirroring the performance of honest nodes. By carefully pacing the injection across 240 rounds and scaling the payload mathematically to the network's local dilution rate, the adversary maximized its effectiveness while remaining completely stealthy across every tested architectural constraint.

6.4 Community Topologies and Boundary Resilience (SBM)

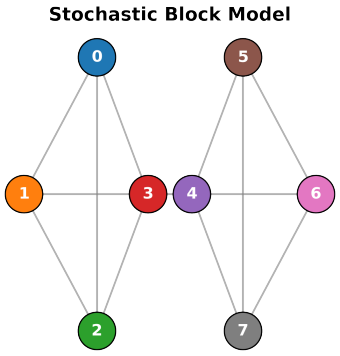
To evaluate the resilience of realistic, community-based networks, we simulated an 8-node Stochastic Block Model (SBM) divided into two distinct communities (Community A: Clients 0-3; Community B: Clients 4-7) connected by a single gateway link between Client 3 and Client 4. The simulation spanned 240 rounds to test long-term boundary resilience.

6.4.1 The Topological Firewall (Peripheral Attacker)

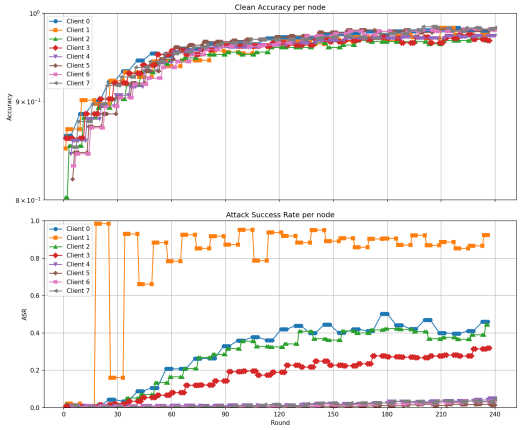
In the first scenario (Figure 6.6b), the attacker is placed at Client 1, deep within Community A. The results demonstrate a clear "echo chamber" effect. Because Client 1 is well-connected locally but isolated from the broader network, the backdoor successfully circulates within Community A, pushing the ASR of its immediate neighbors (Clients 0 and 2) to approximately 0.45 by round 240.

However, the attack entirely fails to cross the community boundary. The ASR for all nodes in Community B (Clients 4, 5, 6, and 7) remains flat at 0.0 for the entire simulation.

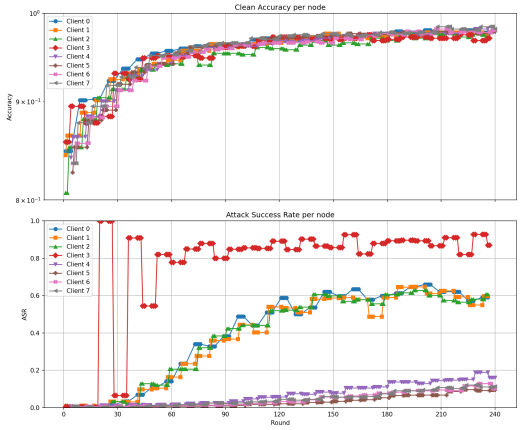
This proves that a sparse gateway link natively functions as a physical, topological firewall. The localized consensus of Community B is too mathematically dense for a peripheral infection to penetrate via a single bridging connection.



(a) Stochastic Block Model Topology



(b) SBM Peripheral Attacker



(c) SBM Gateway Attacker

Figure 6.6: Clean Accuracy and ASR over 240 rounds for the Stochastic Block Model. The top figure illustrates the network topology, with node colors matching the individual client lines in the subsequent plots under two distinct attacker placements.

6.4.2 The Bottleneck Breach Attempt (Gateway Attacker)

To push the network to its absolute limit, the final experiment shifted the compromised node to the structural bottleneck itself. In this scenario (Figure 6.6c), Client 3 (the gateway node of Community A) acts as the adversary, utilizing a higher dynamic boosting factor ($\alpha = d + 1$) commensurate with its higher connectivity.

The change in attacker placement yielded two meaningfull discoveries:

- **Amplified Local Infection:** Because the attacker was positioned at a highly central point within Community A, the local infection rate skyrocketed. Clients 0, 1, and 2 experienced an ASR surge to roughly 0.60, which issignificantly higher than the peripheral attack. Controlling the gateway allowed the adversary to dominate its own community.

Topology	Attacker Placement	Local ASR	Global ASR	Clean Acc.
Fully Connected	Symmetric	0.673	0.673	0.9693
Ring	Symmetric	0.232	0.079	0.9827
Star	Peripheral (Edge)	0.038	0.023	0.9813
Star	Hub (Center)	0.9925	0.9587	0.9933
SBM	Peripheral (Comm A)	0.444	0.0354	0.9813
SBM	Gateway (Comm A)	0.5973	0.1287	0.9786

Table 6.2: Summary of adversarial backdoor metrics at Round 240. Local ASR refers to the infection rate of the attacker's immediate neighbors or local community, while Global ASR refers to distant nodes or secondary communities.

- **The "Consensus Sink" Defense:** Remarkably, despite the attacker having a direct edge to Community B and continuously broadcasting for 240 rounds, the broader network still survived. The ASR for Clients 4 through 7 was heavily suppressed, plateauing below 0.15.

This ultimate failure to bridge the gap highlights the defensive power of Decentralized Federated Learning. When the gateway node of Community B (Client 4) received the highly poisoned parameters from Client 3, it immediately aggregated them with the clean updates from its internal neighbors (Clients 5, 6, and 7).

Community B effectively acted as a massive "consensus sink," structurally washing out the backdoor before it could achieve a viable success rate.

These experiments prove that while continuous, topology-aware backdoor attacks can successfully evade detection and dominate dense local clusters, decentralized community boundaries provide a natural, highly robust defense against global network corruption.

6.5 Validation on Industrial Data (NEU-DET)

To ensure that the observed topological vulnerabilities are not merely an artifact of the relatively simple MNIST dataset, the core experiments were replicated using the NEU-DET dataset. This dataset serves as a complex benchmark for industrial surface defect detection. The objective was to verify whether the structural mechanics of decentralized backdoor propagation hold true in high-stakes engineering applications. In particular, we aimed to test the rapid dissemination in dense networks and the firewall effect of sparse boundaries.

The network was evaluated under two distinct structural extremes. The first is a maximally dense Fully Connected topology, and the second is a community-based Stochastic Block Model (SBM) with a Peripheral Attacker. Across both configurations, the malicious node utilized the continuous $\alpha = d + 1$ topological boosting heuristic. As observed in the prior experiments,

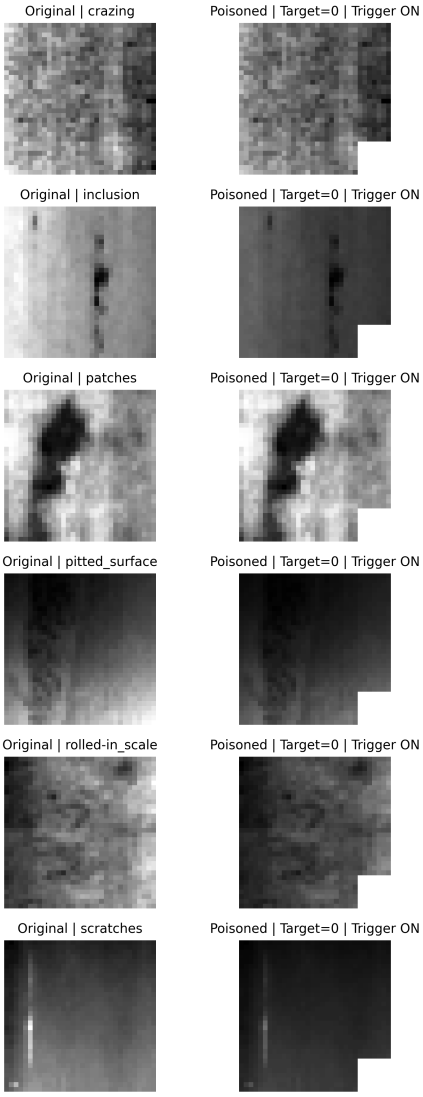


Figure 6.7: Examples of clean and poisoned (triggered) industrial surface defect images from the NEU-DET dataset.

the attacker successfully avoided Catastrophic Forgetting and maintained a Clean Accuracy indistinguishable from the honest majority (Figure 6.8).

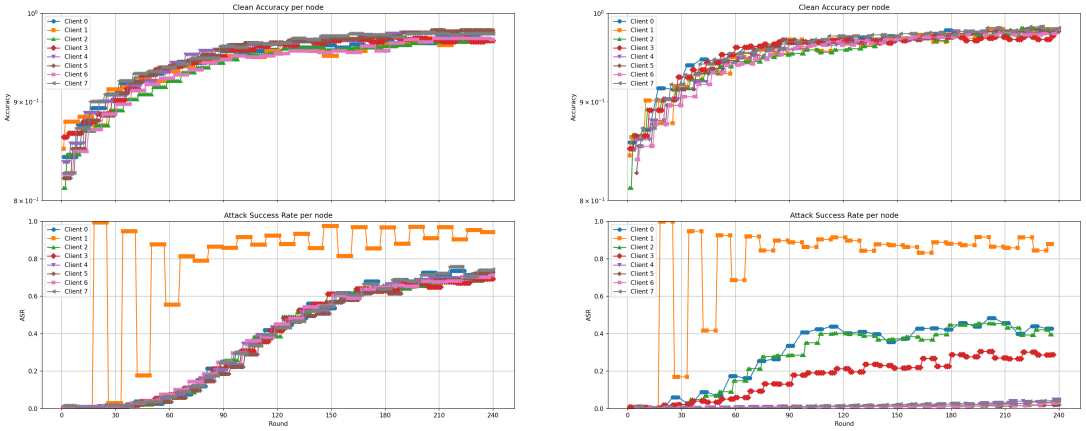
The Attack Success Rate (ASR) confirmed that structural constraints govern the survival of the backdoor regardless of dataset complexity. In the Fully Connected network (Figure 6.8a), the absence of communication bottlenecks allowed the continuous injection to systematically overcome neighborhood dilution. This lack of resistance drove a system-wide infection that reached an ASR of approximately 0.75 by round 240.

Conversely, when the attacker was embedded deep within Community A of the SBM

topology (Figure 6.8b), the network exhibited robust boundary resilience. The malicious payload successfully circulated within the local echo chamber, infecting the immediate neighbors to an ASR of roughly 0.45. However, it entirely failed to penetrate Community B. The single gateway link acted as a natural topological firewall that kept the ASR of the distant community permanently suppressed at 0.0.

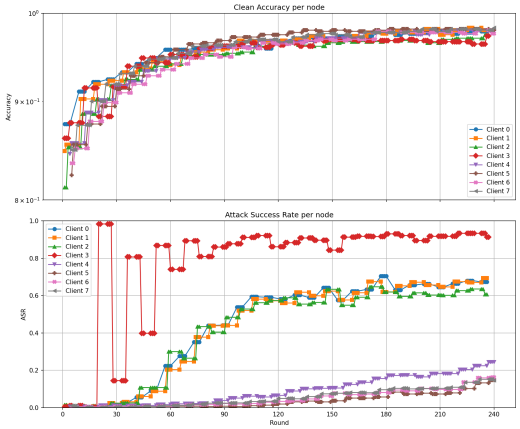
To push the industrial validation to its structural limit, the adversary was finally repositioned to the gateway node itself (Figure 6.8c). While commanding this bottleneck amplified the local infection within Community A (pushing the ASR to nearly 0.70), the broader network remained secure. Community B successfully acted as a consensus sink, filtering out the complex industrial backdoor payload and keeping the distant ASR stagnant below 0.20.

Ultimately, these industrial validations confirm the core hypothesis of this thesis. In Decentralized Federated Learning, the effectiveness of a continuous backdoor attack is dictated by the architectural density of the network rather than the inherent complexity of the training data.



(a) Fully Connected

(b) SBM Peripheral Attacker



(c) SBM Gateway Attacker

Figure 6.8: Clean Accuracy and ASR over 240 rounds for the NEU-DET dataset. The dense network (a) allows a system-wide infection. The sparse SBM topology acts as a topological firewall against peripheral attacks (b), and successfully utilizes Community B as a consensus sink to neutralize attacks originating directly from the structural bottleneck (c).

Eranskina / apéndice

Eranskinak

Bibliography

[1] Brendan McMahan, Ewan Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. *Artificial intelligence and statistics*, pages 1273–1282, 2017. See pages 1, 3.

[2] Róbert Ormándi, István Hegedűs, and Márk Jelasity. Gossip learning with l2-regularized linear models. *Parallel Computing*, 39(12):769–785, 2013. See pages 1, 5.

[3] Qingqian Yang, Peishen Yan, Xiaoyu Wu, Jiaru Zhang, Tao Song, Yang Hua, Hao Wang, Liangliang Wang, and Haibing Guan. Stealthy backdoor attack in federated learning via adaptive layer-wise gradient alignment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 29163–29172, 2025. See pages 1, 9.

[4] Georgios Syros, Gökberk Yar, Simona Boboila, Cristina Nita-Rotaru, and Alina Oprea. Backdoor attacks in peer-to-peer federated learning. *arXiv preprint arXiv:2301.09732*, 2024. See page 1.

[5] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999. See pages 2, 10, and 29.

[6] Adam Piaseczny, Eric Ruzomberka, Rohit Parasnis, and Christopher G Brinton. Adversarial node placement in decentralized federated learning: Maximum spanning-centrality strategy and performance analysis. *IEEE Internet of Things Journal*, 2025. See pages 2, 10.

[7] Ying Bao, Kebin Song, Jiyong Liu, Yan Wang, Yu Yan, and Hongkai Yu. A deep learning-based method for surface defect detection of hot-rolled steel strip. *The International Journal of Advanced Manufacturing Technology*, 102:399–405, 2019. See page 2.

[8] Lin Yuan, Zheng Wang, Lichao Sun, Philip S Yu, and Christopher G Brinton. Decentralized federated learning: A survey and perspective. *IEEE Internet of Things Journal*, 11(21):34617–34638, 2024. See page 4.

[9] István Hegedüs, Gábor Danner, and Márk Jelasity. Gossip learning as a decentralized alternative to federated learning. In *Distributed Applications and Interoperable Systems*, pages 74–90. Springer, 2019. See page 5.

[10] Paul Vanhaesebrouck, Aurélien Bellet, and Marc Tommasi. Decentralized collaborative learning of personalized models over networks. *arXiv preprint arXiv:1710.06175*, 2017. See page 5.

[11] Luigi Palmieri, Chiara Boldrini, Lorenzo Valerio, Andrea Passarella, and Marco Conti. Impact of network topology on the performance of decentralized federated learning. *arXiv preprint arXiv:2402.18606*, 2024. See pages 5, 17.

[12] Ehsan Hallaji, Roozbeh Razavi-Far, Mehrdad Saif, Boyu Wang, and Qiang Yang. Decentralized federated learning: A survey on security and privacy. *IEEE Transactions on Big Data*, 10(2):194–213, 2024. See page 7.

[13] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 1322–1333, 2015. See page 8.

[14] Yuheng Zhang, Ruoxi Jia, Hengrui Pei, Wenxiao Wang, Bo Li, and Dawn Song. The secret revealer: Generative model-inversion attacks against deep neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 253–261, 2020. See page 8.

[15] Giuseppe Ateniese, Luigi V Mancini, Angelo Spognardi, Antonio Villani, Domenico Vitali, and Giovanni Felici. Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers. In *Proceedings of the 26th Annual International Conference on Computer Science and Software Engineering*, pages 137–150, 2015. See page 8.

[16] Karan Ganju, Qi Wang, Wei Yang, Carl A Gunter, and Nikita Borisov. Property inference attacks on fully connected neural networks using permutation invariant representations. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 619–633, 2018. See page 8.

[17] Gyojin Han, Jaehyun Choi, Haehyun Lee, and Junghwan Kim. Reinforcement learning-based black-box model inversion attacks. *arXiv preprint arXiv:2304.04625*, 2023. See page 8.

[18] Nader Bouacida and Prasant Mohapatra. Vulnerabilities in federated learning. *IEEE Access*, 9:63229–63257, 2021. See page 8.

[19] Xiaofeng Xie, Chuanpu Hu, Hao Ren, and Jing Deng. A survey on vulnerability of federated learning: A learning algorithm perspective. *Neurocomputing*, 573:127225, 2024. See page 8.

[20] Guoliang Xia, Jiacheng Chen, Chunhua Yu, and Jianfeng Ma. Poisoning attacks in federated learning: A survey. *IEEE Access*, 11:10708–10722, 2023. See pages 8, 9.

[21] Ali Shafahi, W Ronny Huang, Mahyar Najibi, Octavian Suciu, Christoph Studer, Tudor Dumitras, and Tom Goldstein. Poison frogs! targeted clean-label poisoning attacks on neural networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018. See page 8.

[22] Chen Zhu, W Ronny Huang, Hengduo Li, Gavin Taylor, Christoph Studer, and Tom Goldstein. Transferable clean-label poisoning attacks on deep neural nets. In *Proceedings of the 36th International Conference on Machine Learning*, pages 7614–7623. PMLR, 2019. See page 8.

[23] Eugene Bagdasaryan, Andreas Veit, Yiat Chia, Maxim Fedorov, and Deborah Estrin. How to backdoor federated learning. *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020. See pages 9, 10, and 12.

[24] Chulin Xie, Keli Huang, Pin-Yu Chen, and Bo Li. Dba: Distributed backdoor attacks against federated learning. In *International Conference on Learning Representations (ICLR)*, 2020. See pages 10, 12.

[25] Daniel J Beutel, Taner Topal, Akhavan Akhavanniaz, Nicolas Marti, Yan He, Lorenzo Barkotto, and Nicholas D Lane. Flower: A friendly federated learning framework. *arXiv preprint arXiv:2007.14390*, 2020. See pages 17, 20.

[26] Aitor Belenguer, Jose A. Pascual, and Javier Navaridas. GLow - a novel, Flower-based simulated gossip learning strategy. *arXiv preprint arXiv:2501.10463*, 2025. See page 17.

[27] Ludovico Giaretta and Sarunas Girdzijauskas. Gossip learning: Off the beaten path. *2019 IEEE International Conference on Big Data*, pages 2273–2282, 2019. See page 18.

[28] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social networks*, 5(2):109–137, 1983. See page 19.

[29] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. See page 19.

[30] Jie Mills, Jianwei Hu, and Geyong Min. Communication-efficient federated learning with compensated layer-wise freezing. *IEEE Transactions on Network Science and Engineering*, 7(4):3115–3125, 2019. See page 20.

[31] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, 2014. See page 20.

[32] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. See page 20.

[33] Sheng Sun, Zengqi Zhang, Quyang Pan, Min Liu, Yuwei Wang, Tianliu He, Yali Chen, and Zhiyuan Wu. Staleness-controlled asynchronous federated learning: Accuracy and efficiency tradeoff. *IEEE Transactions on Mobile Computing*, 23(12):12621–12634, 2024. See page 24.